



TrustHire AI

"Campus placement, built on proof."

STUDENT NAME

Rudra Kumar Sharma

ROLL NUMBER

24CSU175

SUBMISSION DATE

July 30, 2026

PROJECT TYPE

5-Day Capstone



<https://bit.ly/Trusthireai>



github.com/ruudrasharma/TrustHire-AI



Running on personal Linux cloud server

TOC Table of Contents

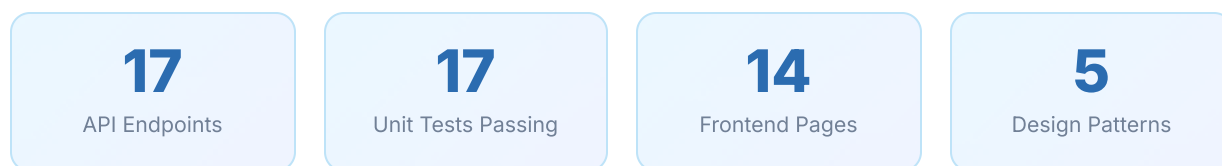
- 1 Project Overview & Key Stats
- 2 System Architecture
- 3 Backend — Spring Boot API
- 4 Design Patterns
- 5 Complete API Reference
- 6 Unit Tests (17/17)
- 7 Frontend — Next.js Web App
- 8 AI Integration (Ollama llama3.2:1b)
- 9 Security Design
- 10 Deployment & Infrastructure
- 11 Screenshots & Evidence
- 12 Repository Structure
- 13 Local Setup & Seed Data
- 14 Known Limitations & Future Work

i Live Deployment

TrustHire AI is deployed on a personal Linux cloud server and publicly accessible at **<https://bit.ly/Trusthireai>**. All three services — frontend, backend, and AI assistant — are running as systemd services with automatic restart on failure or reboot.

1 Project Overview

TrustHire AI is a full-stack campus placement management platform built in 5 days as a capstone project. Unlike conventional CRUD applications, every critical claim in the system is **cryptographically provable** — eligibility decisions are signed, status transitions are hash-chained, and receipts are independently verifiable.



What makes it different

Feature	What it means
Signed Eligibility	Every eligible/ineligible result is HMAC-SHA256 signed. The AI verifies this before use — refuses if tampered.
Hash-Chained Audit	Every status change (SUBMITTED→UNDER_REVIEW→SHORTLISTED→SELECTED) is SHA-256 hash-chained. Reordering or deleting breaks the chain.
Verifiable Receipt	Students download a signed JSON receipt. <code>POST /api/verify</code> validates it statelessly — no database lookup.
Privacy Boundary	Raw CGPA and active backlogs never cross the API boundary — only boolean eligibility and human-readable reasons.
Local LLM Advisory	Ollama (llama3.2:1b) runs on the server CPU. Advisory-only — cannot apply or change any status.

2 System Architecture

Public Internet

<https://bit.ly/Trusthireai>

<https://rudra-ai.tail0fb5d4.ts.net:8443>

|

| HTTPS – TLS terminated by Tailscale Funnel

↓

Linux Cloud Server (Ubuntu)

	Tailscale Funnel	:8443 → localhost:3001	
	Next.js 16 Frontend	(port 3001)	
	systemd: trusthire-frontend.service		
		/api/* catch-all proxy	
	Spring Boot 3 Backend	(port 8081)	
	systemd: trusthire-backend.service		
	Ollama llama3.2:1b	(port 11434)	
	systemd: ollama.service		

Architectural Decisions

Decision	Choice	Rationale
Storage	In-memory <code>ConcurrentHashMap</code>	Repository interface allows zero-change swap to PostgreSQL
API Proxy	Next.js Route Handler <code>/api/[...path]</code>	More reliable than rewrites behind Tailscale Funnel
TLS / Public	Tailscale Funnel	Zero-config HTTPS, no port-forwarding, no cert management
AI Model	Ollama llama3.2:1b	Runs on server CPU, no API key, 1.3 GB, verified working
Process Mgmt	systemd	Auto-restart on crash/reboot, full environment isolation
Signing	HMAC-SHA256	Standard, deterministic, produces hex digest
Audit chain	SHA-256 linking	Tamper-evident without external trust anchor
SQL vs NoSQL	PostgreSQL (production)	FK relationships (drive→company, app→student/drive) need relational guarantees
CAP trade-off	Writes: CP · Listings: AP	Status changes must be correct; drive listings tolerate stale

3 Backend — Spring Boot API

Technology Stack

Component	Details
Language	Java 17
Framework	Spring Boot 3.3 (Web, Validation)
Build	Maven 3.9
Cryptography	<code>javax.crypto</code> — HMAC-SHA256 + SHA-256
AI client	<code>java.net.http.HttpClient</code> → Ollama REST API
Storage	<code>ConcurrentHashMap</code> (in-memory, no database)
Port	8081 (production) · 8080 (dev default)

Package Structure

```
com.trusthire.ai/
├── domain/      Student, Company, PlacementDrive, Application,
                  ApplicationStatus (state machine), AuditEvent, EligibilityCriteria
├── repository/  StudentRepository (interface) + InMemoryStudentRepository
                  CompanyRepository, DriveRepository, ApplicationRepository
├── policy/      Criterion (interface) ← Strategy pattern
                  CgpaCriterion, BacklogCriterion, SkillCriterion,
                  ProgrammeCriterion, GraduationYearCriterion
                  CompositeEligibilityPolicy ← Composite pattern
├── security/    ResultSigner    (HMAC-SHA256)
                  HashChain     (SHA-256 chain build + verify)
├── chat/        ChatClient (interface) ← Adapter pattern
                  OllamaChatClient
├── service/     StudentService, CompanyService, DriveService,
                  EligibilityService, ApplicationService,
                  AuditTrailService, ReceiptService, ChatService
├── web/         StudentController, CompanyController, DriveController,
                  ApplicationController, VerifyController, ChatController,
                  GlobalExceptionHandler, dto/
├── exception/   ResourceNotFoundException, DuplicateResourceException,
                  InvalidStatusTransitionException, DeadlinePassedException,
                  EligibilityViolationException, ChatServiceUnavailableException
```

4 Design Patterns

4.1 Strategy — Eligibility Evaluation

Each eligibility rule is a pluggable strategy. Adding a new criterion requires no changes to any existing class.

```
interface Criterion {
    CriterionResult evaluate(Student student, PlacementDrive drive);
}

// 5 concrete strategies – each independently testable
class CgpaCriterion implements Criterion { ... }
class BacklogCriterion implements Criterion { ... }
class SkillCriterion implements Criterion { ... }
class ProgrammeCriterion implements Criterion { ... }
class GraduationYearCriterion implements Criterion { ... }
```

4.2 Composite — Policy Aggregation

Runs all criteria, aggregates pass/fail and reason strings, returns a signed result.

```
class CompositeEligibilityPolicy {
    private final List<Criterion> criteria;

    EligibilityResult evaluate(Student student, PlacementDrive drive) {
        // Aggregates all criteria results → returns signed EligibilityResult
    }
}
```

4.3 Adapter — AI Chat Isolation

The service layer calls a clean `ChatClient` interface — it never knows about Ollama's JSON shape.


```

interface ChatClient {
    String chat(String prompt);    // clean, testable interface
}

class OllamaChatClient implements ChatClient {
    // Adapts Ollama's { model, prompt, stream } JSON to the simple interface
    // Throws ChatServiceUnavailableException if Ollama is unreachable
}

```

4.4 State Machine — Application Status

Guards all transitions at the domain level — invalid transitions are rejected with a typed exception before any database write.

```

enum ApplicationStatus {
    SUBMITTED, UNDER_REVIEW, SHORTLISTED, SELECTED, REJECTED;

    boolean canTransitionTo(ApplicationStatus next) {
        return switch (this) {
            case SUBMITTED    → next = UNDER_REVIEW || next = REJECTED;
            case UNDER_REVIEW → next = SHORTLISTED  || next = REJECTED;
            case SHORTLISTED  → next = SELECTED      || next = REJECTED;
            default           → false;    // SELECTED/REJECTED are terminal
        };
    }
}

```

4.5 Repository — Storage Abstraction

All 8 services depend only on repository interfaces. Swapping the in-memory implementation for JPA + PostgreSQL requires changing one Spring `@Bean` and zero service classes.

```

class ApplicationService {
    private final ApplicationRepository repo;    // interface only – never InMemory
    // Swap InMemory → PostgreSQL = change one @Configuration class,
    // ApplicationService stays identical
}

```

5 Complete API Reference



Base URL

Production: <https://rudra-ai.tail0fb5d4.ts.net:8443/api> | Short:
<https://bit.ly/Trusthireaapi>

Students

Method	Endpoint	Status	Description
POST	/api/students	201, 400, 409	Create student profile
GET	/api/students/{id}	200, 404	Get student by ID
PUT	/api/students/{id}	200, 400, 404	Update student profile
GET	/api/students/{id}/applications	200, 404	Student's applications

```
// POST /api/students - Request
{ "name":"Aditi Rao", "email":"aditi@example.edu", "programme":"B.Tech CSE",
  "graduationYear":2027, "cgpa":8.1, "activeBacklogs":0,
  "skills":["Java","Spring Boot"] }

// Response 201 - Note: cgpa and activeBacklogs NEVER returned (privacy boundary)
{ "id":"STU-001", "name":"Aditi Rao", "email":"aditi@example.edu",
  "programme":"B.Tech CSE", "graduationYear":2027, "skills":["Java","Spring Boot"] }
```

Companies

Method	Endpoint	Status	Description
POST	/api/companies	201, 400, 409	Create company
GET	/api/companies	200	List all companies
GET	/api/companies/{id}	200, 404	Get company

Drives

Method	Endpoint	Status	Description
POST	/api/drives	201, 400, 404	Create placement drive
GET	/api/drives	200	List / filter drives
GET	/api/drives/{id}	200, 404	Get drive
GET	/api/drives/{driveId}/eligibility/{studentId}	200, 404	Signed eligibility check

```
// GET /api/drives/{driveId}/eligibility/{studentId} – Eligible response
{ "eligible":true, "reasons":["Meets all eligibility criteria for this drive"],
  "signature":"95804b2eaa699eb0041d8ab9902de38396c8280d74d1baf50d17297c6f441863" }

// Ineligible response – reasons list shows exactly which criteria failed
{ "eligible":false,
  "reasons":["Active backlogs exceed the maximum allowed (0)",
            "CGPA does not meet the minimum requirement of 7.5",
            "Missing required skill(s): Java, Spring Boot"],
  "signature":"89f3878b91b9145473aa4d29630ed0e6bba4bf1830d381765a87d2abed312cfa" }
```

Applications

Method	Endpoint	Status	Description
POST	/api/drives/{driveId}/applications	201, 400, 404, 409	Apply to drive
GET	/api/applications	200	All applications (coordinator)
GET	/api/applications/{id}	200, 404	Get application
PATCH	/api/applications/{id}/status	200, 400, 404	Update status (triggers audit event)
GET	/api/applications/{id}/audit	200, 404	Hash-chained audit trail
GET	/api/applications/{id}/receipt	200, 404	Issue signed receipt

```
// GET /api/applications/{id}/audit – hash chain (event[n].prevHash = event[n-1].hash)
[ { "fromStatus":"SUBMITTED","toStatus":"UNDER_REVIEW",
    "prevHash":"06beb0f1a8a439b91137440d374255390c4dcc013cf684afa348b04bebf93a49",
    "hash":    "50dc761879976c9e233c59131dc3ad84524baf30bb31a0c6508d83f91b0b" },
  { "fromStatus":"UNDER_REVIEW","toStatus":"SHORTLISTED",
    "prevHash":"50dc761879976c9e233c59131dc3ad84524baf30bb31a0c6508d83f91b0b",
    "hash":    "1101fc66716953839de73548cb7db24d2dc7942a504c5ddfd1510c0fef07a8c6" }
]
```

Receipt & Verify

Method	Endpoint	Status	Description
GET	/api/applications/{id}/receipt	200, 404	Issue signed receipt
POST	/api/verify	200 always	Stateless receipt verification

```
// Unmodified receipt → { "valid":true, "reason":null }
// Tampered status → { "valid":false, "reason":"Signature mismatch – receipt may have been tampered with" }
```

AI Chat

Method	Endpoint	Status	Description
POST	/api/chat	200, 400, 503	AI career assistant (Ollama)

```
// Request
{ "studentId":"STU-001", "message":"What skills do I need for backend?", "intent":"faq" }

// Response 200 – verified working on live server
{ "answer":"To become a backend engineer: Java/Python/JS, Spring Boot, databases,
    Cloud platforms, Docker, OOP, Git, testing frameworks...",
  "model":"llama3.2:1b", "advisory":true }

// Response 503 – graceful degradation when Ollama is unreachable
{ "status":503, "code":"CHAT_SERVICE_UNAVAILABLE",
  "message":"AI assistant temporarily unavailable. All other features remain functional." }
```

Standard Error Contract

```
{ "timestamp":"2026-07-30T12:52:06.777951Z", "status":409,  
  "code":"DUPLICATE_RESOURCE",  
  "message":"Student STU-004 has already applied to drive DRV-001",  
  "path":"/api/drives/DRV-001/applications" }  
  
// Error codes: DUPLICATE_RESOURCE · RESOURCE_NOT_FOUND · INVALID_STATUS_TRANSITION  
//              DEADLINE_PASSED · CHAT_SERVICE_UNAVAILABLE · BAD_REQUEST
```

6 Unit Tests — 17/17 Passing



All tests pass

`cd backend && mvn test` → Tests run: 17, Failures: 0, Errors: 0, Skipped: 0

Test Class	Scenarios Covered	Count
<code>ApplicationStatusTest</code>	All valid transitions (SUBMITTED→UNDER_REVIEW, UNDER_REVIEW→SHORTLISTED, SHORTLISTED→SELECTED, any→REJECTED) + all invalid rejections (SELECTED→any, REJECTED→any, backward transitions)	~7
<code>CompositeEligibilityPolicyTest</code>	Eligible pass (all criteria met), each criterion blocking individually (CGPA, backlogs, skills, programme, grad year), privacy boundary (no CGPA in output)	~6
<code>ResultSignerTest</code>	Sign/verify round-trip passes, tamper detection (modified eligible field fails), tamper detection (modified reasons fail)	~2
<code>ApplicationServiceTest</code>	Full apply lifecycle, duplicate application rejection (409), hash chain integrity after transitions	~2

7 Frontend — Next.js Web App

Technology Stack

Component	Details
Framework	Next.js 16.2 (App Router)
Language	TypeScript
Styling	Vanilla CSS with CSS custom properties (design tokens)
Animation	Framer Motion (spring physics)
Icons	Lucide React
Port	3001 (production) · 3000 (development)

All 14 Pages

Role	Page	API Called
–	Landing / role selector	–
Student	Student Home — live open drives feed	GET /api/drives
Student	Profile create / update	POST/PUT /api/students/{id}
Student	Browse + search + filter drives	GET /api/drives?role=&location=
Student	Drive detail + eligibility check + Apply	GET /api/drives/{id}/eligibility/{sid}
Student	My applications list	GET /api/students/{id}/applications
Student	Application detail + ProofRail + HashChainTimeline	GET /api/applications/{id}/audit
Student	Verifiable receipt download + verify	GET ../receipt + POST /api/verify
Student	AI Career Assistant chat	POST /api/chat
Coordinator	Coordinator dashboard	GET /api/drives + GET /api/applications
Coordinator	Manage companies	POST/GET /api/companies
Coordinator	Manage drives	POST/GET /api/drives
Coordinator	Review + update application statuses	GET/PATCH /api/applications/{id}/status
System	API catch-all proxy (/api/[...path]/route.ts)	Forwards all /api/* to Spring Boot :8081

Key Components

Component	Purpose
<code>OrbitDock</code>	Animated floating navigation dock (student section, Framer Motion)
<code>ProofRail</code>	Sidebar showing live cryptographic proof state — signed eligibility, chain status, receipt validity
<code>HashChainTimeline</code>	Visual timeline of hash-chained audit events with linked hash display
<code>ReceiptCard</code>	Formatted verifiable receipt with download + one-click verify button
<code>api/[...path]/route.ts</code>	Server-side proxy — all <code>/api/*</code> requests forwarded to Spring Boot

API Client Design (lib/api.ts)

```
// Single typed module — no raw fetch() anywhere else in the codebase
const BASE = process.env.NEXT_PUBLIC_API_BASE_URL ?? "";
// In production: BASE="" → /api/... → Route Handler → Spring Boot

export const drivesApi = {
  getAll: () ⇒ request<DriveResponse[]>("/api/drives"),
  getById: (id) ⇒ request<DriveResponse>(`/api/drives/${id}`),
  checkEligibility: (driveId, sid) ⇒
    request<EligibilityResponse>(`/api/drives/${driveId}/eligibility/${sid}`),
  apply: (driveId, sid) ⇒
    request<ApplicationResponse>(`/api/drives/${driveId}/applications`,
      { method: "POST", body: { studentId: sid } }),
};
```

8 AI Integration

8.1 Setup on Linux Server

```
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl enable ollama
ollama pull llama3.2:1b    # 1.3 GB – runs on server CPU, no GPU needed
```

8.2 Chat Flow

Step	Component	Action
1	ChatController	Validates <code>intent</code> field (faq / eligibility / preparation / profile)
2	ChatService	Builds contextual prompt based on intent + student/drive data
3	OllamaChatClient	<code>POST localhost:11434/api/generate</code> with model + prompt
4	ChatController	Returns <code>{ answer, model:"llama3.2:1b", advisory:true }</code>
—	GlobalExceptionHandler	If Ollama down: <code>503 CHAT_SERVICE_UNAVAILABLE</code> — all other APIs unaffected

8.3 Intent Types

Intent	Prompt includes
<code>faq</code>	General placement guidance only
<code>eligibility</code>	Student's signed eligibility result for specific drive (signature re-verified first)
<code>preparation</code>	Drive role + required skills list
<code>profile</code>	Student programme + current skills



Advisory Boundary — Strictly Enforced

The AI assistant can explain, suggest, and advise. It **cannot** apply on behalf of a student, change any application status, or make any decisions. Every response carries `"advisory": true`. This is enforced at the service layer — not just documented.

9 Security Design

9.1 Signed Eligibility Results (HMAC-SHA256)

```
signature = HMAC-SHA256(  
  key    = TRUSTHIRE_SIGNING_SECRET (env var, never hardcoded),  
  input  = eligible.toString() + "|" + reasons.join("|") + studentId + driveId  
)  
  
// ChatService re-verifies this signature before using eligibility data  
// If tampered: refuses to proceed, returns error
```

9.2 Hash-Chained Audit Trail (SHA-256)

```
Event1.hash      = SHA256(fields1 + "GENESIS_HASH")  
Event2.prevHash = Event1.hash  
Event2.hash      = SHA256(fields2 + Event2.prevHash)  
  
// Chain property: deleting, inserting, or reordering any event  
// invalidates every subsequent hash – detectable without external trust anchor
```

9.3 Verifiable Receipt (Stateless Verification)

```

Receipt.signature = HMAC-SHA256(
    applicationId + studentId + driveId + status + chainTipHash + issuedAt
)

// POST /api/verify:
//   → Recomputes signature from receipt fields
//   → { "valid":true } if matches
//   → { "valid":false, "reason":"Signature mismatch" } if tampered
//   → NO database call required – purely mathematical

```

9.4 Privacy Boundary

`Student.cgpa` and `Student.activeBacklogs` are internal domain fields only. The `StudentResponse` DTO and every other response object deliberately excludes them. Only boolean eligibility and human-readable reason strings are exposed over the API.

9.5 Unique Features Summary

Feature	Implementation	Test Evidence
Signed Eligibility	<code>ResultSigner.java</code> + <code>EligibilityService</code>	<code>signed-eligibility-result.png</code>
Hash-Chained Audit	<code>HashChain.java</code> + <code>AuditTrailService</code>	<code>hash-chain-audit.png</code>
Stateless Receipt Verify	<code>ReceiptService</code> + <code>POST /api/verify</code>	<code>receipt-verify-pass/fail.png</code>
Privacy Boundary	<code>StudentResponse</code> DTO omits CGPA/backlogs	All student API responses
Graceful AI Degradation	<code>ChatServiceUnavailableException</code> → 503	Other APIs return 200/201 when Ollama down

10 Deployment & Infrastructure

10.1 Server Configuration

Item	Value
OS	Ubuntu Linux (personal cloud server)
Public URL	https://rudra-ai.tail0fb5d4.ts.net:8443
Short link	https://bit.ly/Trusthireai
Java	OpenJDK 17
Node.js	v20.20.2 (via nvm)
AI Model	Ollama llama3.2:1b (1.3 GB, CPU-only)
TLS	Tailscale Funnel (automatic, no cert management)
Funnel mapping	:8443 → localhost:3001

10.2 systemd Services

```
# trusthire-backend.service
[Service]
WorkingDirectory=/home/rudra/TrustHire-AI/backend
ExecStart=/usr/bin/java -jar target/trusthire-ai-*.jar
EnvironmentFile=/etc/trusthire.env
Restart=always
RestartSec=5

# trusthire-frontend.service
[Service]
WorkingDirectory=/home/rudra/TrustHire-AI/frontend
ExecStart=/home/rudra/.nvm/versions/node/v20.20.2/bin/node \
    node_modules/.bin/next start -p 3001
EnvironmentFile=/etc/trusthire.env
Restart=always

# Both: sudo systemctl enable <service> → auto-start on reboot
```

10.3 Environment Variables

```
# /etc/trusthire.env (not in version control)
TRUSTHIRE_SIGNING_SECRET=<random-32-byte-hex>
TRUSTHIRE_CORS_ORIGIN=https://rudra-ai.tail0fb5d4.ts.net:8443
TRUSTHIRE_OLLAMA_URL=http://localhost:11434
TRUSTHIRE_OLLAMA_MODEL=llama3.2:1b
```

10.4 Update Procedure

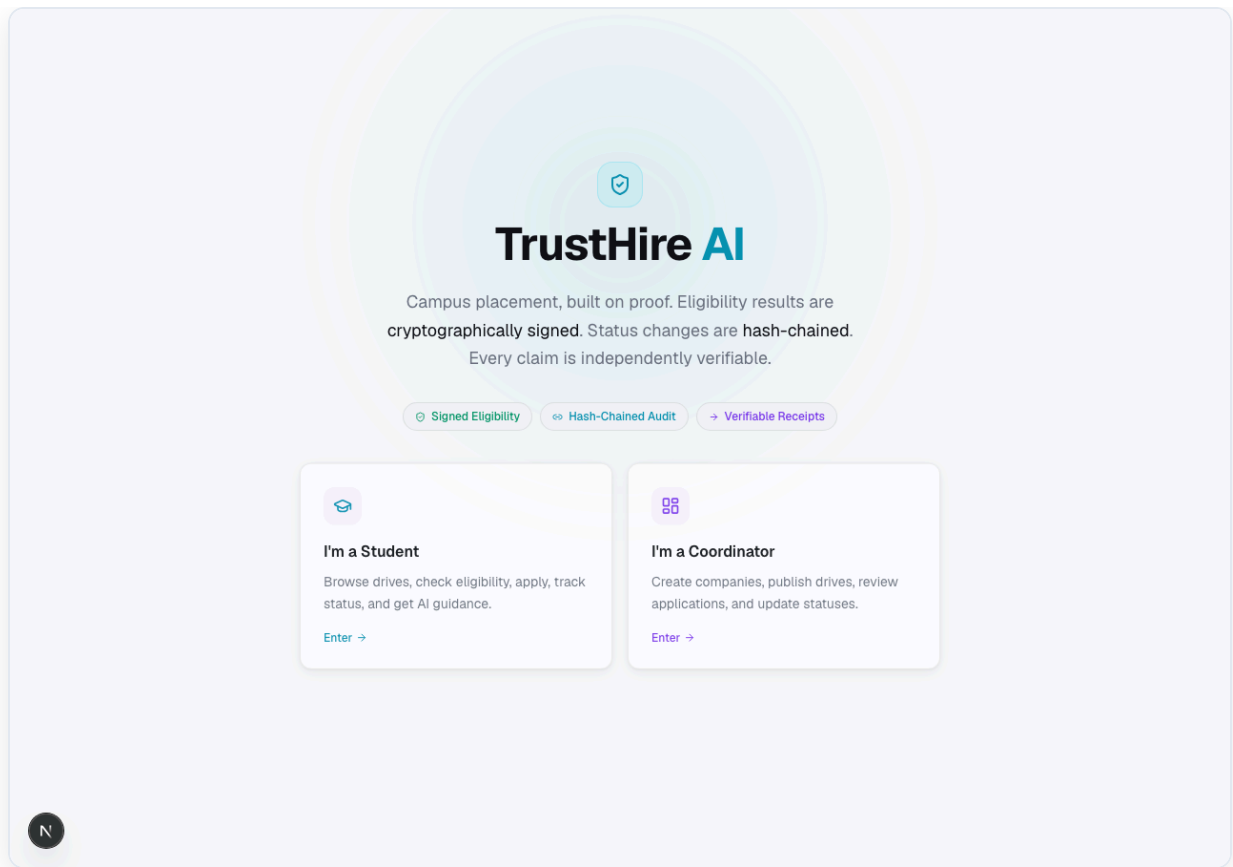
```
cd ~/TrustHire-AI && git pull origin main

# If backend changed:
cd backend && mvn clean package -DskipTests -q
sudo systemctl restart trusthire-backend

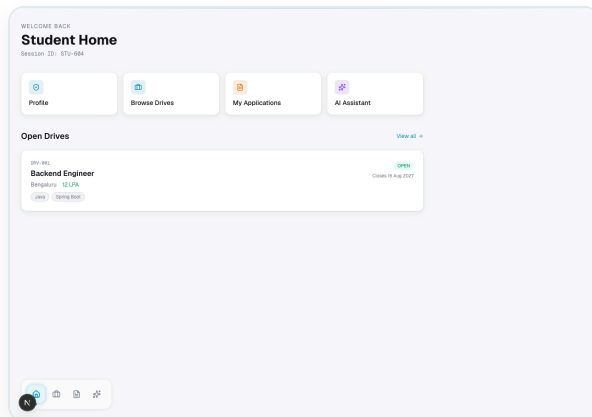
# If frontend changed:
cd frontend && npm run build
sudo systemctl restart trusthire-frontend
```

11 Screenshots & Evidence

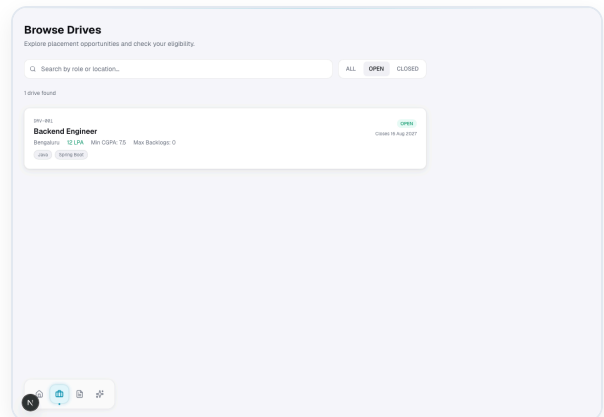
Landing Page



Landing page — role selector with animated feature badges

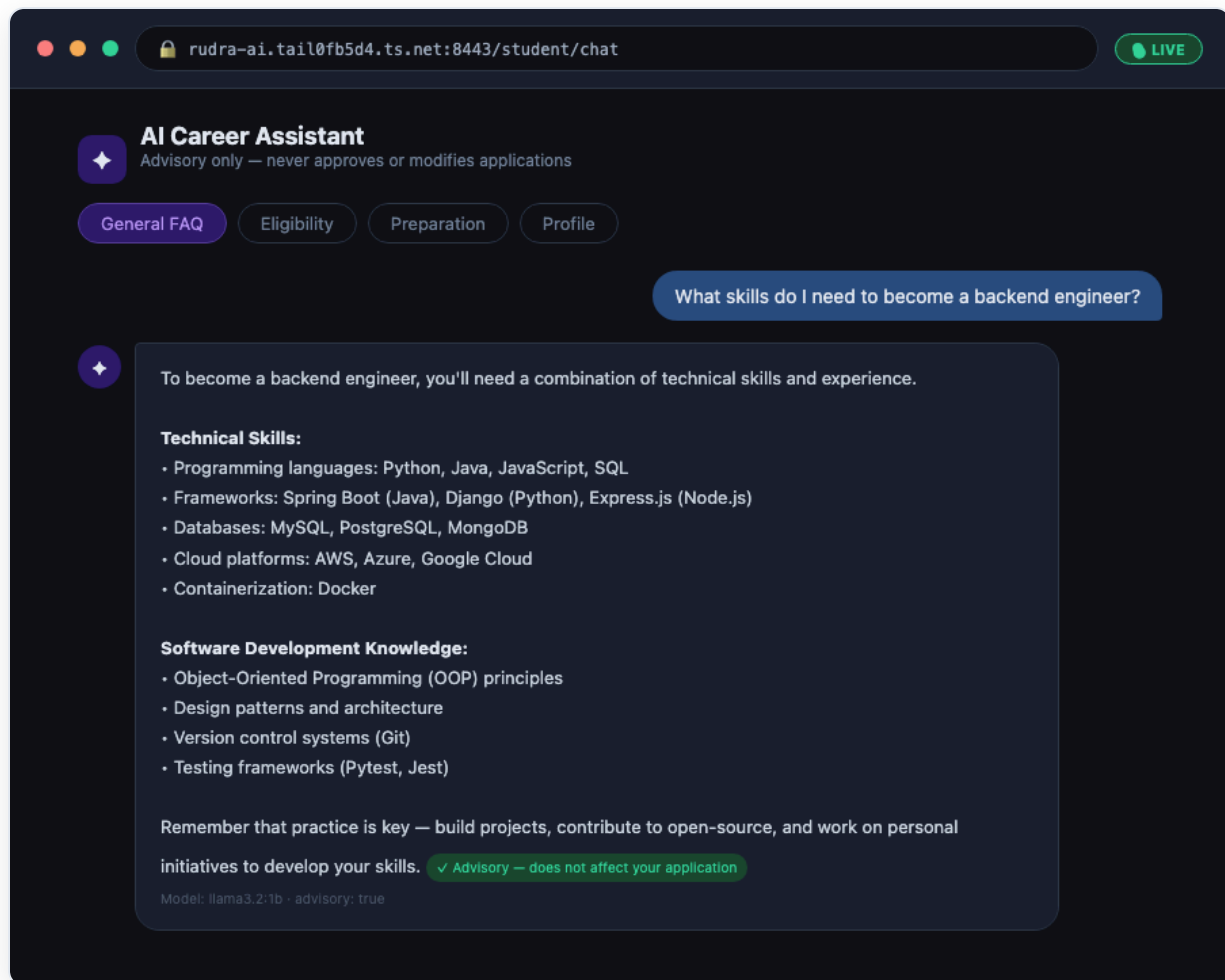


Student Home — live Backend Engineer drive from seeded data



Browse Drives — search + filter interface

AI Chat — Live Ollama Response



AI Career Assistant — real llama3.2:1b response via live Tailscale URL · advisory:true · model shown

Live Deployment Proof

TrustHire AI — Live Deployment

Running on Linux cloud server via Tailscale Funnel

Public URL **https://rudra-ai.tail0fb5d4.ts.net:8443**

Frontend **● Active**

Next.js 16 · port 3001 · systemd managed **RUNNING**

Backend API **● Active**

Spring Boot 3 · port 8081 · systemd managed **RUNNING**

AI Assistant (Ollama) **● Active**

llama3.2:1b · port 11434 · response verified ✓ **RESPONDING**

Live API response — POST /api/chat (via Tailscale)

```
{
  "answer": "To become a backend engineer, you'll need: Java/Python/JS,
            Spring Boot / Django / Express.js, Databases (MySQL, MongoDB),
            Cloud (AWS/GCP), Docker, OOP, Git, and testing frameworks...",
  "model": "llama3.2:1b",
  "advisory": true
}
```

Tailscale Funnel **ACTIVE**

Port mapping **:8443 → localhost:3001**

Auto-restart on reboot **systemd enabled ✓**

Live server proof — all 3 services running · real API response from Ollama · Tailscale Funnel active

Cryptographic Proof Screenshots

GET /api/drives/DRV-001/eligibility(studentid) — Signed Results

```
STU-001 — ADITI RAO (CGPA 8.1, JAVA + SPRING BOOT)
ELIGIBLE
{
  "eligible": true,
  "reasons": ["Meets all eligibility criteria for this drive"],
  "signature": "9508462aaa699e0041d8ab9902de38396c6288d74d1ba758d17297c6f441863"
}
```

```
STU-002 — ROB LOW (CGPA 6.0, 2 BACKLOGS, PYTHON ONLY)
INELIGIBLE
{
  "eligible": false,
  "reasons": [
    "Active backlogs exceed the maximum allowed (0)",
    "CGPA does not meet the minimum requirement of 7.5",
    "Missing required skill(s): Java, Spring Boot"
  ],
  "signature": "89f3878b91b9145473aa4d28638ed0e0ba4b71830d381765a87d2abed312cfa"
}
```

Note: raw CGPA/backlogs never appear in responses — privacy boundary enforced

GET /API/APPLICATIONS/APP-001/AUDIT — HASH-CHAINED EVENTS

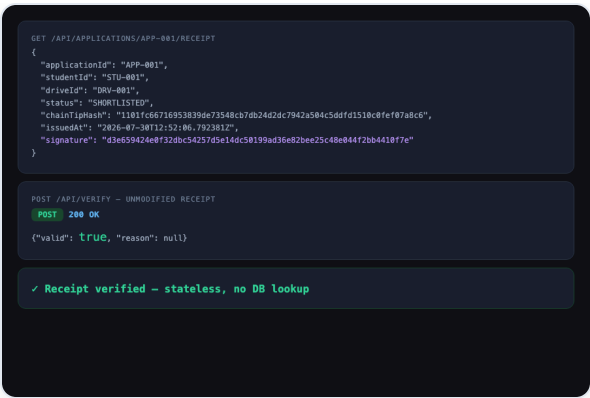
```
GET 200 OK
{
  [
    {
      "applicationId": "APP-001",
      "fromStatus": "SUBMITTED",
      "toStatus": "UNDER_REVIEW",
      "timestamp": "2026-07-30T12:35:05.062675Z",
      "prevHash": "86beb0f1a8a439b91137440d374255390c4dccc013cf684afa348b04bebf93a49",
      "hash": "58dc761879976c9e23c59131dc3ad04524aafef30bb31a0cc6508d03f910b0"
    },
    {
      "applicationId": "APP-001",
      "fromStatus": "UNDER_REVIEW",
      "toStatus": "SHORTLISTED",
      "timestamp": "2026-07-30T12:35:05.091312Z",
      "prevHash": "58dc761879976c9e23c59131dc3ad04524aafef30bb31a0cc6508d03f910b0",
      "hash": "1101fc6716953839de73548cb7db24d2dc7942a504c5ddf1d1510c0fe07a0cd"
    }
  ]
}
```

Chain verified

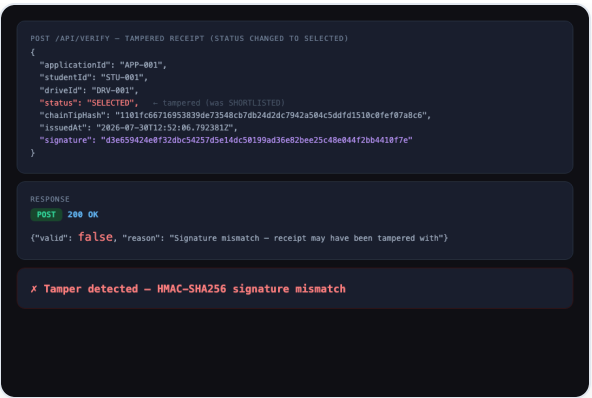
event[1].prevHash == event[0].hash — deleting or reordering any event breaks the chain

Signed eligibility — eligible (CGPA 8.1) + ineligible (CGPA 6.0) with HMAC-SHA256 signatures

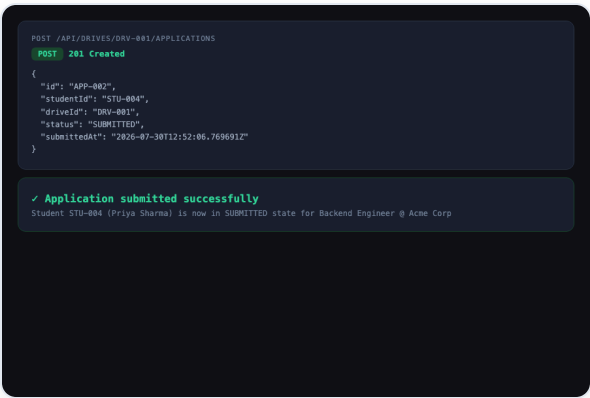
Hash-chained audit — event[1].prevHash == event[0].hash proven



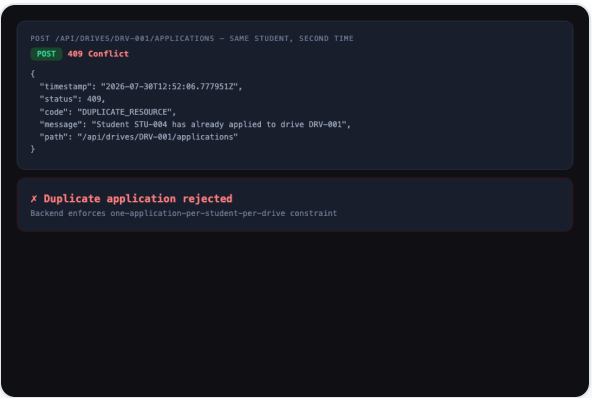
Receipt verify — valid:true on unmodified receipt (stateless)



Tamper detection — valid:false when status field modified

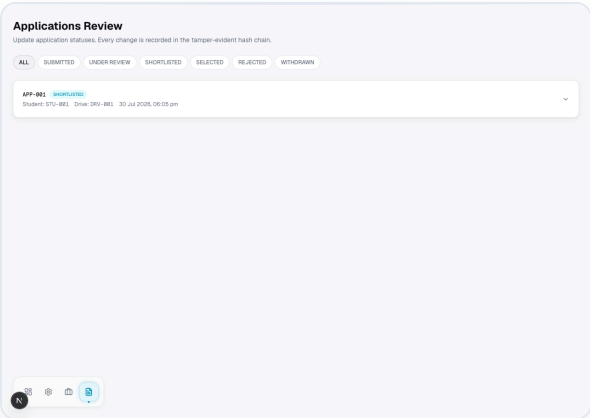


Successful application — 201 Created (T-06 evidence)

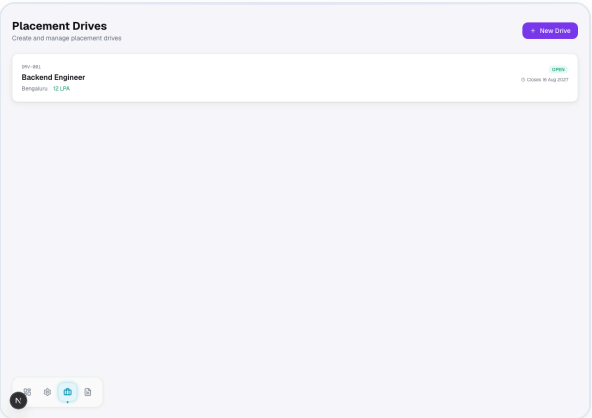


Duplicate application rejected — 409 Conflict (T-07 evidence)

Coordinator Dashboard



Coordinator applications — status controls visible



Coordinator drives — create + list interface

12 Repository Structure

```
TrustHire-AI/                                github.com/ruudrasharma/TrustHire-AI
├─ README.md                                Main documentation with all sections
├─ backend/                                Spring Boot API
│   ├── pom.xml
│   └─ src/
│       ├── main/java/com/trusthire/ai/
│       └─ test/java/com/trusthire/ai/
│           ├── domain/ApplicationStatusTest.java
│           ├── policy/CompositeEligibilityPolicyTest.java
│           ├── security/ResultSignerTest.java
│           └─ service/ApplicationServiceTest.java
├─ frontend/                                Next.js web app
│   ├── package.json
│   ├── next.config.ts
│   ├── .env.production                     NEXT_PUBLIC_API_BASE_URL=""
│   └─ src/
│       ├── app/                            14 pages (App Router)
│       └─ components/                      OrbitDock, ProofRail, HashChainTimeline, Rece
iptCard
│   └─ lib/                                api.ts, types.ts, utils.ts
├─ docs/
│   ├── requirements-and-assumptions.md
│   ├── hld-architecture.md
│   ├── lld-class-diagram.md
│   ├── sequence-apply.md
│   ├── sequence-chat.md
│   ├── api-contract.md
│   ├── checkpoint-log.md
│   └─ test-evidence/README.md             T-01 → T-21 checklist
├─ postman/
│   └─ TrustHireAI.postman_collection.json  21 test scenarios, port 8081
└─ screenshots/                            18 real evidence screenshots
```

13 Local Setup & Seed Data

```
git clone https://github.com/ruudrasharma/TrustHire-AI.git
cd TrustHire-AI

# Backend (Terminal 1)
cd backend && mvn spring-boot:run
# API available at http://localhost:8081

# Frontend (Terminal 2)
cd frontend && npm install && npm run dev
# App available at http://localhost:3000

# AI Chat (Terminal 3 – optional)
ollama serve && ollama pull llama3.2:1b
```

Seed Data (or import postman/TrustHireAI.postman_collection.json)

```
BASE=http://localhost:8081

# 1. Create eligible student
curl -X POST $BASE/api/students -H "Content-Type: application/json" \
  -d '{"name":"Aditi Rao","email":"aditi@example.edu","programme":"B.Tech CSE",
    "graduationYear":2027,"cgpa":8.1,"activeBacklogs":0,"skills":["Java","Spring
    Boot"]}'

# 2. Create company
curl -X POST $BASE/api/companies -H "Content-Type: application/json" \
  -d '{"name":"Acme Corp","sector":"Fintech","description":"Leading fintech compan
  y"}'

# 3. Create drive (note: deadline must have Z suffix for Instant.parse)
curl -X POST $BASE/api/drives -H "Content-Type: application/json" \
  -d '{"companyId":"CMP-001","role":"Backend Engineer","location":"Bengaluru",
    "packageOffered":"12 LPA","deadline":"2027-08-15T23:59:59Z",
    "requiredSkills":["Java","Spring Boot"],"minCgpa":7.5,
    "maxActiveBacklogs":0,"eligibleProgrammes":["B.Tech CSE"],"minGraduationYea
    r":2026}'
```

14 Known Limitations & Future Work

Current Limitations (Intentional — per project scope)

Limitation	Reason
No authentication	Demonstration scope — no JWT/OAuth/sessions by design
No persistent storage	Data resets on restart — in-memory by design
AI quality bounded by model size	Llama3.2:1b is 1.3GB — no fine-tuning or RAG
Audit chain not externally anchored	Proves internal consistency; not anchored to external ledger

Zero-Service-Layer Upgrade Path

Upgrade	What changes
PostgreSQL persistence	Swap <code>InMemoryXxxRepository</code> → JPA implementations. Services: unchanged.
OAuth2 / JWT auth	Add Spring Security resource server. Role guards per controller.
Redis caching	Cache drive listings + FAQ responses (TTL 60s, invalidate on write).
External chain anchoring	Periodically write chain tip hash to a public transparency log.